



UNIVERSIDAD SAN IGNACIO DE LOYOLA
VICERRECTORADO ACADÉMICO
FACULTAD DE INGENIERÍA Y TECNOLOGÍA
CARRERA DE INGENIERÍA DE SOFTWARE

SÍLABO

I. DATOS GENERALES

- 1.1. Nombre del curso: Programación de Sistemas
- 1.2. Código: ISW-41203
- 1.3. Carrera: Ingeniería de Software
- 1.4. Ciclo: VI
- 1.5. Créditos: 4
- 1.6. Horas semanales: 5 horas (3 Teoría, 2 Práctica)
- 1.7. Modalidad: Presencial/Semipresencial
- 1.8. Semestre Académico: 2024-02
- 1.9. Pre-requisito: Estructuras de Datos y Algoritmos

II. SUMILLA

El curso de Programación de Sistemas es de naturaleza teórico-práctica y pertenece al área de formación especializada de la carrera de Ingeniería de Software. El propósito del curso es que el estudiante comprenda la interacción entre el software y el hardware, enfocándose en el manejo de memoria y la optimización de bajo nivel. Los temas principales incluyen: arquitectura de sistemas desde la perspectiva del programador, gestión de memoria dinámica y estática, punteros y aritmética de direcciones, jerarquía de memoria (cache), optimización de código para el rendimiento del CPU, y programación en lenguajes de sistemas (C/C++ y nociones de Assembly).

III. COMPETENCIAS

3.1. Competencia General:

- Diseño y desarrollo de sistemas: Capacidad para diseñar y desarrollar componentes de software de sistema que optimicen el uso de los recursos del hardware.

3.2. Competencias Específicas:

- Implementa soluciones de software eficientes utilizando lenguajes de bajo nivel para el control directo de recursos.
- Analiza y resuelve problemas relacionados con la gestión de memoria (leaks, fragmentación, punteros colgantes).
- Aplica técnicas de optimización de código para mejorar el tiempo de ejecución y la eficiencia energética de las aplicaciones.

IV. UNIDADES DE APRENDIZAJE

UNIDAD 1: Fundamentos y Representación de Datos

- Introducción a la programación de sistemas y entorno de desarrollo (Toolchain).
- Representación de datos a nivel de bits: enteros, punto flotante y operaciones bitwise.
- El modelo de compilación: Preprocesador, compilador, ensamblador y linker.



UNIDAD 2: Manejo de Memoria

- El espacio de direccionamiento del proceso: Stack vs. Heap.
- Gestión de memoria dinámica: malloc, free, calloc, realloc.
- Punteros, arreglos y aritmética de direcciones avanzada.
- Detección de errores de memoria (Memory leaks, Buffer overflows) y herramientas de depuración (Valgrind, GDB).

UNIDAD 3: Arquitectura y Optimización de Código

- La jerarquía de memoria y el impacto de la memoria Cache (L1, L2, L3).
- Localidad espacial y temporal de los datos.
- Optimización de ciclos y transformaciones de código para mejorar el pipeline del CPU.
- Introducción a la vectorización (SIMD) e instrucciones intrínsecas.

UNIDAD 4: Interfaz con el Sistema y Ensamblador

- Introducción al lenguaje ensamblador (x86-64 o ARM).
- Interfaz entre C y Assembly: Convenciones de llamada (Calling conventions).
- Llamadas al sistema (System Calls) y gestión de archivos a bajo nivel.
- Herramientas de perfilado (Profiling) para identificar cuellos de botella.

V. METODOLOGÍA

El curso se desarrolla mediante:

- Sesiones teóricas: Exposición de conceptos con ejemplos reales de sistemas operativos.
- Laboratorios guiados: Resolución de ejercicios de programación en C y Assembly.
- Proyectos de curso: Implementación de un gestor de memoria propio o un optimizador de procesamiento de datos.
- Aprendizaje basado en problemas: Análisis de casos de fallos de sistemas por mala gestión de memoria.

VI. SISTEMA DE EVALUACIÓN

La nota final (NF) se obtiene mediante el siguiente promedio ponderado:

- Evaluación Permanente (EP): 40% (Prácticas calificadas, laboratorios y participación).
- Examen Parcial (EA): 25% (Evaluación teórica-práctica de unidades 1 y 2).
- Proyecto Final (TF): 35% (Desarrollo de un software de sistema optimizado).

VII. BIBLIOGRAFÍA

- Bryant, R. E., & O'Hallaron, D. R. (2016). Computer Systems: A Programmer's Perspective (3rd ed.). Pearson.
- Kernighan, B. W., & Ritchie, D. M. (1988). The C Programming Language (2nd ed.). Prentice Hall.
- Love, R. (2013). Linux System Programming: Talking Directly to the Kernel and C Library. O'Reilly Media.
- Fog, A. (2021). Optimizing software in C++: An optimization guide for Windows, Linux, and Mac platforms. Technical University of Denmark.